

Revisions After the Proposal Defense

8 October 2026 · Alexandra Sulit, Willard Soriano, Steven Evian Lozano · Accompanies the revised proposal, *SynapseOS: Designing and Evaluating a Conversational Interface Layer for Personal Computing* (Chapters 1–3, 118 pages). Page numbers below refer to that revision.

This report sets out how each concern raised at the proposal defense, and one raised in an earlier review, has been addressed, and where the change can be found. Section 2 summarizes the new evidence, Section 3 lists every change to the proposal, and Section 4 states the limitations we identified in the process. The appendices provide supporting material and are not required reading.

1. Response to the Panel

#	RAISED BY	CONCERN	RESPONSE	LOCATION
1	Panel, proposal defense	Chapter 1 has no Background of the Study.	The background, problem statement, objectives, contributions, significance and scope were present, but written as continuous prose without subsection headings. Chapter 1 now has numbered subsections, as Chapters 2 and 3 do: 1.1 Background of the Study, 1.2 Statement of the Problem, 1.3 Objectives of the Study, 1.4 Contributions of the Study, 1.5 Significance of the Study, and 1.6 Scope and Limitations of the Study. No content was removed.	Ch. 1, pp. 1-13 (1.1 p. 1; 1.2 p. 4; 1.3 p. 6; 1.4 p. 7; 1.5 p. 10; 1.6 p. 11)
2	Panel, proposal defense	How is SynapseOS a computer science study rather than an IT project?	The study’s core contribution is an algorithm, not the system built around it: a static program analysis that derives a shell command’s effects on files from its syntax, resolves targets hidden behind wrappers or computed at run time, and computes the minimal capture needed to undo the command. It belongs to the tradition of abstract interpretation and type-and-effect systems, and is evaluated independently of the interface on 777 held-out commands with ground truth obtained by sandboxed execution. Three new results strengthen this: the ablation in row 3, a validation of the baseline against open-source tools, and an end-to-end verification (Section 2).	Algorithm: §2.1(c), p. 60 Theory: Ch. 2, p. 23 Evaluation: §3.1, p. 72 Baseline validity: §3.1(c), p. 76 Verification: §2.3, p. 68
3	Adviser, earlier review	How much of the performance comes from the algorithm itself, and how much from the manually constructed rule tables?	Measured by ablation. With the identical rule table but the algorithm’s composition disabled, capture coverage on the held-out corpus falls from 89.5% to 35.4% (McNemar’s exact test, $p = 2.1 \times 10^{-53}$; risk difference +0.542, 95% CI [+0.484, +0.593]). Silent loss is 0% in both configurations. The rule table alone recovers roughly a third of what the full algorithm recovers.	§3.1(h) and Table 3.5, pp. 82–83 Summarized in §1.4, p. 9

In one sentence: the background was present but unlabeled and is now a numbered section; and the computer-science contribution, an algorithm, is now supported by measured evidence that the algorithm itself, rather than its rule table or a weak baseline, is responsible for the results.

2. Summary of New Evidence

2.1 The algorithm, not its rule table, produces the result

The adviser’s earlier question was answered only in prose before the defense. It is now measured: disabling composition (the algorithm’s ability to see through searches, pipes, loops and wrappers to the files a command will touch) while keeping the same rule table reduces data-recovery coverage from 89.5% to 35.4% on the 777-command held-out corpus. The loss is concentrated in commands whose targets are hidden or determined at run time, exactly where composition operates. (§3.1(h), Table 3.5, pp. 82–83.)

2.2 The baseline is stricter than the tools in use today

Because the pattern-list baseline was written by the team, it was compared with command-safety checks that others ship, scored on the same corpus and ground truth. On the 325 commands that destroy files:

SYSTEM	DESTRUCTIVE COMMANDS RUN WITH NO CONFIRMATION OR BACKUP	HARMLESS COMMANDS NEEDLESSLY QUESTIONED
OpenAI Codex CLI (dangerous-command check)	96.6%	0.3%
cc-safety-net (Standard / Strict / Paranoid)	80.6% / 79.1% / 72.6%	1.9% / 3.2% / 4.0%
SynapseOS pattern list (the proposal’s baseline)	30.8%	8.1%
Recoverability analysis	0.0%	8.6%

The proposal’s baseline is therefore the strictest list tested, and comparisons against it understate the analysis’s advantage. Both external tools are designed for coding agents working in version-controlled projects, so these figures describe how they would serve a person’s own files, not their intended purpose. No shipping tool determines a command’s effects before it runs in order to capture what it would destroy; this is the gap identified in §2.4. (§3.1(c), p. 76; Appendix B.)

2.3 The analysis’s decisions carry through to real files

Ten test cases were run through the complete system, with the local language model writing the commands, once with the analysis disabled and once with it enabled, inspecting the files in the initial state, after execution, and after undo. Where a command’s targets were indirect, undo restored the files in every case with the analysis and in none without it; without it, two data-destroying commands ran with no confirmation at all; where targets were named directly, both configurations behaved identically; and where an operation cannot be undone, the analysis required confirmation and said so. The verification is illustrative and supports no claim about rates; those rest on the corpus. (§2.3, p. 68; Appendix C.)

3. Changes to the Proposal

LOCATION	CHANGE	PAGE
Chapter 1	Numbered subsections 1.1–1.6	1–13
§1.4 Contributions	One sentence reporting the composition ablation	9
§2.3 Validation	One paragraph reporting the manual verification	68
§3.1(c) Baselines	One paragraph reporting the comparison with open-source checks	76
§3.1(h), Table 3.5	New subsection: composition ablation	82–83
§3.6 Threats to validity	New entry: unscoped undo	90
References	Two software references: Codex CLI; cc-safety-net	93, 95

The proposal grew from 113 to 118 pages. Nothing was removed, and no previously reported figure changed.

4. Limitations Identified

The work above surfaced the following, which we report rather than leave to be found.

- **Undo is not scoped to the current context.** It reverts the most recent recorded change, even one made in another folder or an earlier session. It shows what it will revert and asks first, but correct use depends on the user reading that preview. Before data collection, undo will either be restricted to the current folder and session or reset for each session. (§3.6, p. 90.)
- **The local language model mistranslates loosely worded requests.** In the manual verification, the 3B model wrote the intended command for only two of seven loosely worded requests, mostly by mishandling file names that contain spaces; the requests were then made explicit. This is the translation limitation already stated in §1.6 (p. 11) and does not affect the analysis, which judges whatever command it is given.
- **The manual verification is illustrative.** Its cases were chosen to show specific behaviours, so no rate is claimed from it; the claims of RQ1 rest on the corpus evaluation.
- **The external tools have a different purpose.** They protect coding projects under version control, not personal files, so the comparison shows that current lists do not protect a desktop user’s files, not that the tools are poor at their own task.
- **A defect in the baseline-only configuration.** With the analysis disabled, a redirect to `/dev/null` is mistaken for a file to back up and suppresses the warning that a deletion cannot be undone. It does not occur with the analysis enabled, which is the default.

Appendices

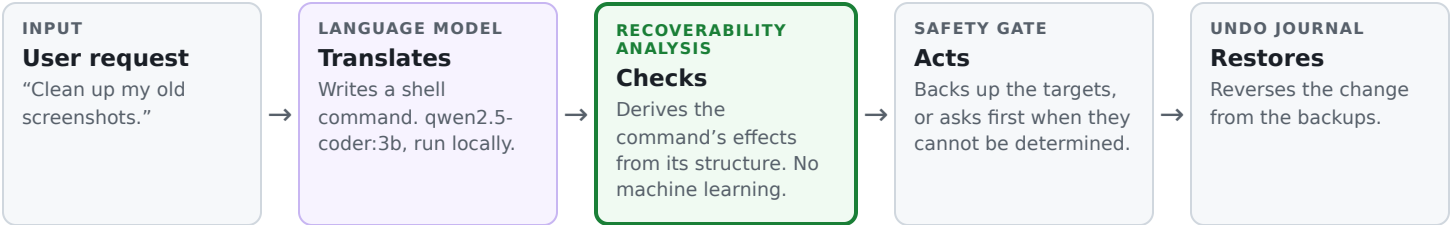
APPENDIX	CONTENT	LENGTH
A	How the recoverability analysis works: where it sits relative to the language model, and its five steps on one example	1 page
B	Comparison with open-source command-safety checks: sources, licences, method, results	2 pages
C	Manual verification report: ten test cases, recorded with and without the analysis	12 pages

All figures are reproducible from the project repository: the corpus, the per-command verdicts of every system, the analysis scripts, and the session logs of the manual verification.

How the Recoverability Analysis Works

SynapseOS has two separate components. A local language model translates a request into a shell command; the recoverability analysis then decides, before that command runs, what it will change and whether it can be undone. **The analysis does not use the language model:** it is deterministic program analysis, specified in Chapter 3, §2.1(c) (p. 60).

Where the analysis sits



Probabilistic: the same request may yield different commands. **Deterministic:** the same command always yields the same decision.

Inside the analysis, in five steps

Running example: `find . -name "Screenshot*.png" -delete`

1

Parse
Reads the command's structure with a shell-language parser.

Program `find`; search path `.`; filter `-name "Screenshot*.png"`; action `-delete`.

2

Look up
A rule table records what each program does to files.

`find ... -delete` removes every file that matches its filter; which files those are is not named in the command.

3

Compose
Sees through searches, pipes, loops and wrappers to the program that acts.

The deletion is carried out by `find` itself on files it discovers while running. The same reasoning applies to `find -exec rm`, `| xargs rm`, `for` loops and `sh -c`.

4

Resolve
Runs only the proven read-only part, in a sandbox where nothing can be written.

ILLUSTRATION (SIMPLIFIED) OF AN INTERNAL STEP; USERS NEVER SEE IT:

```
$ find . -name "Screenshot*.png" # -delete replaced by a read-only print
./Screenshot 2026-09-20 184502.png
./Screenshot 2026-09-14 101211.png
```

5

Decide
Classifies the effects and plans what to back up.

ACTUAL ANALYSIS OUTPUT, UNEDITED:

```
command: find . -name "Screenshot*.png" -delete
composition: enabled (full algorithm)
class: recoverable-with-capture
remove ./Screenshot 2026-09-20 184502.png
remove ./Screenshot 2026-09-14 101211.png
```

Both files are backed up before the command runs. Had their effects been undeterminable (an unknown program, inline Python, `shred`), the command would be classified *unrecoverable* and confirmation required.

Why the separation matters

It cannot be talked into anything
The safety decision is made by code, not a model. It judges the command as actually written, so a flawed or unexpected command from the model is judged on its real effects.

It is independent of the command's source
A command from the model, a person, or a script is treated identically. This is why the 777-command evaluation (§3.1, p. 72) required no model at all.

It is the computer-science contribution
The analysis is static program analysis in the tradition of abstract interpretation (p. 23), not machine learning. The model is the interface; the analysis is the research.

Comparison with Open-Source Command-Safety Checks

Purpose

The proposal compares the recoverability analysis with an enumerated-pattern list written by the research team. To test whether that baseline is representative rather than conveniently weak, two command-safety checks that others ship with AI agents were scored on the same 777 held-out commands, against the same ground truth obtained by sandboxed execution.

Systems compared

SYSTEM	SOURCE AND VERSION	LICENCE	WHAT IT CHECKS
Codex CLI	OpenAI, <code>openai/codex</code> , commit 1f4c473 (1 September 2026)	Apache-2.0	On Linux and macOS, forced deletion only (<code>rm -f</code> , <code>rm -rf</code>), including behind <code>sudo</code> , <code>env</code> , <code>trap</code> and nested <code>bash -c</code>
cc-safety-net	<code>kenryu42/cc-safety-net</code> , version 2.6.1; used with Claude Code, Codex, Gemini CLI and eleven other agents	MIT	Destructive file and version-control operations; three presets (Standard, Strict, Paranoid)

A third candidate, `destructive_command_guard` , was excluded: its licence withholds all rights, including use in an evaluation harness, from OpenAI, Anthropic and anyone acting on their behalf, and an Anthropic model carried out part of this work. Codex’s former allow-list of known-safe commands no longer exists in its current source, so only what Codex currently ships was used.

Method

- **Codex CLI** was ported from its source, function by function, and the port was verified against Codex’s own unit tests, all of which pass. These include forced deletion inside conditionals, loops, command substitutions, traps and nested shells, and cases that must not be flagged. Each command was evaluated as Codex receives a model’s shell call.
- **cc-safety-net** was run unmodified through its own exported checking function, once per command under each preset, from the command’s own starting directory. Its answers were confirmed not to depend on where that directory is located.
- **Scoring** was identical to the proposal’s: a silent loss is a command that destroys data and runs with neither a confirmation nor a backup; a needless confirmation is one asked of a command that loses nothing. Neither external tool backs anything up, so neither is scored on recovery.

Results

The 80 commands with effects outside the file system (remote sessions, mounts, signals) are excluded below, because the external tools do not claim to cover them. On the remaining 325 file-destroying commands, and the 372 commands that lose nothing:

SYSTEM	SILENT LOSS (95% CI)	NEEDLESS CONFIRMATION
Codex CLI	314/325 = 96.6% (94–98%)	1/372 = 0.3%
cc-safety-net, Standard	262/325 = 80.6% (76–85%)	7/372 = 1.9%
cc-safety-net, Strict	257/325 = 79.1% (74–83%)	12/372 = 3.2%
cc-safety-net, Paranoid	236/325 = 72.6% (68–77%)	15/372 = 4.0%
SynapseOS pattern list (proposal’s baseline)	100/325 = 30.8% (26–36%)	30/372 = 8.1%
Fail-closed list	0/325 = 0.0% (0–1%)	101/372 = 27.2%
Recoverability analysis	0/325 = 0.0% (0–1%)	32/372 = 8.6%

Including the 80 commands outside the file system, silent loss is 97% for Codex CLI, 78–84% for cc-safety-net, 41% for the pattern list, and 0% for the analysis.

Interpretation

- **The proposal's baseline is the strictest list tested.** It lets 31% of file-destroying commands through, where the open-source checks let 73–97% through, so comparing the analysis with it understates the analysis's advantage over lists in current use.
- **The external tools were designed for a different purpose.** They guard coding agents working inside version-controlled projects, where deleting project files is routine and version control is the safeguard; they block catastrophic operations rather than everyday deletions. Even in its Paranoid preset, cc-safety-net permits `rm notes.txt` and `find ... -exec rm`. The figures show that current lists do not protect a person's own files, not that these tools perform poorly at their intended task.
- **Recovery has no external counterpart.** Neither tool determines what a command will destroy before it runs, and neither makes any provision for undoing it. No shipping tool was found that does; that is the gap the proposal addresses (§2.4).
- **The analysis reaches the strictest protection without its cost.** It matches the fail-closed list's zero silent loss while asking needlessly about as often as the pattern list (8.6% against 8.1%), not three times as often.
- **Codex also sandboxes commands,** which limits where they can write. That is a separate mechanism from a list and is not part of this comparison.

Reproducibility

The Codex port and its tests, the script that calls cc-safety-net, every per-command verdict (3,108 lines), and the scoring scripts are in the project repository, with exact commands to reproduce the tables (`docs/external-baselines.md`).

Manual Verification of the Recoverability Analysis

Recorded 2026-10-08 · SynapseOS prototype · Language model: qwen2.5-coder:3b (local) · Resolver sandbox: bubblewrap

1. Purpose and Scope

This report records a manual verification of the recoverability analysis (Chapter 3, §2.1(c)). Each test case is executed twice through the deployed system, once with the analysis disabled and once with it enabled, and the affected files are inspected in the initial state, after execution, and after `synapse undo`. It complements the corpus evaluation (§3.1), which presents commands to the analysis directly, by showing the analysis's effect on real files. The procedure is specified in `prototype/manualtest/STORIES.md`.

2. Method

Part A — Paired execution (TC-01 to TC-07). Each request is sent to the deployed system, where the local language model writes the command, and the run is repeated with the analysis disabled. Each run starts from a freshly created test directory and an empty undo journal. Requests name the kind of command under test; where the model's first attempt was malformed or of a different shape, the request was restated to give the command explicitly (TC-03 to TC-06). In TC-07 the model omitted the quotes a file name containing a space requires, even when given the exact command, so the target was changed from `beach copy.jpg` to `installer.zip`; the property tested, a directly named target, is unchanged. With loose phrasing the model produced a usable command of the intended shape in two of seven cases, a translation limitation outside the analysis's scope. In every reported case the model wrote the same command in both runs. **TC-08** examines the analysis alone, as composition cannot be disabled in the deployed system. **Part B — Everyday phrasing (TC-09, TC-10).** A plainly worded request, with the same evidence.

Recording protocol. Each run took place in a real terminal session; its text output was saved as a log and drawn as the image shown. In TC-01 to TC-08, lines beginning `$` are labels printed by the session script to show the command that ran, and lines beginning `#` are explanatory comments added afterwards; every other line is program output, unaltered. Outcomes were read from the logs, which are provided alongside this report. Deletions were approved while target files remained and declined thereafter, and an undo was approved only if its preview restored the target files. All runs used an isolated home directory. A result of *Pass* denotes that observed behaviour matched expected behaviour, including expected failures of the baseline.

3. Summary of Results

CASE	PROPERTY VERIFIED	WITHOUT THE ALGORITHM	WITH THE ALGORITHM	RESULT
TC-01	Run-time target resolution	Confirmed; not restored	Confirmed; restored	Pass
TC-02	Resolution through an execution wrapper	Confirmed; not restored	Confirmed; restored	Pass
TC-03	Opaque interpreter code	No confirmation; not restored	Confirmed with warning; not restored	Pass
TC-04	Metadata modification outside the baseline list	No confirmation; not restored	Confirmed; restored	Pass
TC-05	Loop expansion	Confirmed; not restored	Confirmed; restored	Pass
TC-06	Inherently irrecoverable operation	Confirmed; not restored	Confirmed; not restored	Pass
TC-07	Explicitly named target (control case)	Confirmed; restored	Confirmed; restored	Pass
TC-08	Composition ablation (analysis only)	With composition disabled, targets are no longer identified		Pass
TC-09	Natural-language request, language model in the loop	Confirmed; not restored	Confirmed; restored	Pass
TC-10	Undo journal scope	Undo proposes the most recent entry regardless of context (known limitation)		Pass

PART A — PAIRED EXECUTION

TC-01 Run-time target resolution

Pass

Request “use find with -delete to remove the files whose names start with Screenshot” → model wrote `find . -name 'Screenshot*' -delete` in both runs. Evidence: `ls Screenshot*` in the initial state, after execution, and after undo.

Observation The model wrote the same command in both runs. The targets exist only while `find` runs: without the analysis they could not be restored, and with it undo restored both.

WITHOUT THE ALGORITHM `SYNAPSE_ANALYSIS=off`

```
# Initial state: which files are there?
$ ls Screenshot*
'Screenshot 2026-09-14 101211.png' 'Screenshot 2026-09-20 184502.png'

# Ask SynapseOS, algorithm OFF: the AI writes the command, then it runs
$ SYNAPSE_ANALYSIS=off synapse "use find with -delete to remove the file
s whose names start with Screenshot"
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: find . -name 'Screenshot*' -delete
stats: 25 tokens in 11.415s
blocked: find . -name 'Screenshot*' -delete is irreversible – find -dele
te (find -delete removes every matching file with no built-in undo, and
the matches are only known while it runs)
WARNING: This command decides what to delete while it runs, so I cannot
copy anything first. Undo will NOT be able to bring it back.
If you want the safety net, name the files or folders directly instead
.
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
stats: 14 tokens in 5.619s
All files starting with "Screenshot" have been deleted.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls Screenshot*
ls: cannot access 'Screenshot*': No such file or directory
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: find . -name 'Screenshot*' -delete
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:09:47+08:00)
note: 2 change(s) from this command could not be safely reconstructed
and are not part of this undo: [Screenshot 2026-09-14 101211.png Screenshot
2026-09-20 184502.png]
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls Screenshot*
ls: cannot access 'Screenshot*': No such file or directory
```

WITH THE ALGORITHM `default configuration`

```
# Initial state: which files are there?
$ ls Screenshot*
'Screenshot 2026-09-14 101211.png' 'Screenshot 2026-09-20 184502.png'

# Ask SynapseOS, algorithm ON: the AI writes the command, then it runs
$ synapse "use find with -delete to remove the files whose names start w
ith Screenshot"
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: find . -name 'Screenshot*' -delete
stats: 25 tokens in 4.829s
blocked: find . -name 'Screenshot*' -delete is irreversible – find -dele
te (find -delete removes every matching file with no built-in undo, and
the matches are only known while it runs)
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
stats: 14 tokens in 5.533s
All files starting with "Screenshot" have been deleted.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls Screenshot*
ls: cannot access 'Screenshot*': No such file or directory
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: find . -name 'Screenshot*' -delete
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:10:05+08:00)
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/S
creenshot 2026-09-14 101211.png
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/S
creenshot 2026-09-20 184502.png
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls Screenshot*
'Screenshot 2026-09-14 101211.png' 'Screenshot 2026-09-20 184502.png'
```

TC-02 Resolution through an execution wrapper

Pass

Request "use find with -exec rm to delete every file with copy in its name" → model wrote `find . -type f -name '*copy*' -exec rm -f {} +` in both runs. Evidence: `ls *copy*` in the initial state, after execution, and after undo.

Observation The same command in both runs. The deletion is performed by a command embedded in `find`; only with the analysis did undo restore the copies.

WITHOUT THE ALGORITHM `SYNAPSE_ANALYSIS=off`

```
# Initial state: which files are there?
$ ls *copy*
'beach copy.jpg' 'sunset copy.jpg'

# Ask SynapseOS, algorithm OFF: the AI writes the command, then it runs
$ SYNAPSE_ANALYSIS=off synapse "use find with -exec rm to delete every f
ile with copy in its name"
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: find . -type f -name '*copy*' -exec rm -f {} +
stats: 33 tokens in 7.283s
blocked: find . -type f -name '*copy*' -exec rm -f {} + is irreversible
- rm (rm can permanently delete files with no built-in undo)
WARNING: This command decides what to delete while it runs, so I cannot
copy anything first. Undo will NOT be able to bring it back.
If you want the safety net, name the files or folders directly instead
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
stats: 14 tokens in 5.835s
All files with "copy" in their name have been deleted.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls *copy*
ls: cannot access '*copy*': No such file or directory
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: find . -type f -name '*copy*' -exec rm -f {} +
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:10:21+08:00)
note: 2 change(s) from this command could not be safely reconstructed
and are not part of this undo: [beach copy.jpg sunset copy.jpg]
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls *copy*
ls: cannot access '*copy*': No such file or directory
```

WITH THE ALGORITHM default configuration

```
# Initial state: which files are there?
$ ls *copy*
'beach copy.jpg' 'sunset copy.jpg'

# Ask SynapseOS, algorithm ON: the AI writes the command, then it runs
$ synapse "use find with -exec rm to delete every file with copy in its
name"
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: find . -type f -name '*copy*' -exec rm -f {} +
stats: 33 tokens in 5.96s
blocked: find . -type f -name '*copy*' -exec rm -f {} + is irreversible
- rm (rm can permanently delete files with no built-in undo)
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
stats: 14 tokens in 5.832s
All files with "copy" in their name have been deleted.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls *copy*
ls: cannot access '*copy*': No such file or directory
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: find . -type f -name '*copy*' -exec rm -f {} +
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:10:40+08:00)
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/b
each copy.jpg
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/s
unset copy.jpg
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls *copy*
'beach copy.jpg' 'sunset copy.jpg'
```

TC-03 Opaque interpreter code

Pass

- Request** "run this exact command: python3 -c "import os; os.remove('notes.txt')"" → model wrote `python3 -c "import os; os.remove('notes.txt')"` in both runs. Evidence: `ls notes.txt` in the initial state, after execution, and after undo.
- Observation** **Without the analysis the command ran with no confirmation** and the file was lost. With it, confirmation was required, with a warning that undo could not restore the file. The file is lost in both runs; the analysis turns a silent loss into an informed decision.

WITHOUT THE ALGORITHM SYNAPSE_ANALYSIS=off

```
# Initial state: which files are there?
$ ls notes.txt
notes.txt

# Ask SynapseOS, algorithm OFF: the AI writes the command, then it runs
$ SYNAPSE_ANALYSIS=off synapse 'run this exact command: python3 -c "import os; os.remove('notes.txt')"'
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: python3 -c "import os; os.remove('notes.txt')"
stats: 28 tokens in 6.707s
exit code: 0
```

```
step 2: DONE
stats: 14 tokens in 6.31s
The command was run successfully, and notes.txt has been deleted.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls notes.txt
ls: cannot access 'notes.txt': No such file or directory
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: python3 -c "import os; os.remove('notes.txt')"
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:18:40+08:00)
note: 1 change(s) from this command could not be safely reconstructed
and are not part of this undo: [notes.txt]
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls notes.txt
ls: cannot access 'notes.txt': No such file or directory
```

WITH THE ALGORITHM default configuration

```
# Initial state: which files are there?
$ ls notes.txt
notes.txt

# Ask SynapseOS, algorithm ON: the AI writes the command, then it runs
$ synapse 'run this exact command: python3 -c "import os; os.remove('notes.txt')"'
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: python3 -c "import os; os.remove('notes.txt')"
stats: 28 tokens in 5.551s
blocked: python3 -c "import os; os.remove('notes.txt')" needs confirmation – opaque: python3 runs arbitrary code; its effects are not modelled
WARNING: This command decides what to delete while it runs, so I cannot
copy anything first. Undo will NOT be able to bring it back.
If you want the safety net, name the files or folders directly instead
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
stats: 14 tokens in 6.135s
The command python3 -c "import os; os.remove('notes.txt')" was executed
successfully, and the file notes.txt was removed.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls notes.txt
ls: cannot access 'notes.txt': No such file or directory
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: python3 -c "import os; os.remove('notes.txt')"
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:18:59+08:00)
note: 1 change(s) from this command could not be safely reconstructed
and are not part of this undo: [notes.txt]
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls notes.txt
ls: cannot access 'notes.txt': No such file or directory
```

TC-04 Metadata modification outside the baseline list

Pass

Request “run this exact command: find Projects -type f -exec chmod 600 {} +” → model wrote `find Projects -type f -exec chmod 600 {} +` in both runs. Evidence: `ls -go Projects` in the initial state, after execution, and after undo.

Observation **Without the analysis the command ran with no confirmation** and undo reported nothing to undo; `run.sh` stayed non-executable. With it, prior permissions were captured and undo restored both files.

WITHOUT THE ALGORITHM SYNAPSE_ANALYSIS=off

```
# Initial state: which files are there?
$ ls -go Projects
total 8
-rw-r--r-- 1 12 Oct  8 20:19 app.py
-rwxr-xr-x 1 21 Oct  8 20:19 run.sh

# Ask SynapseOS, algorithm OFF: the AI writes the command, then it runs
$ SYNAPSE_ANALYSIS=off synapse 'run this exact command: find Projects -t
ype f -exec chmod 600 {} +'
model: qwen2.5-coder:3b  endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: find Projects -type f -exec chmod 600 {} +
      stats: 27 tokens in 4.063s
exit code: 0
```

```
step 2: DONE
      stats: 14 tokens in 6.296s
The command `find Projects -type f -exec chmod 600 {} +` was executed su
ccessfully, and the exit code was 0.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls -go Projects
total 8
-rw----- 1 12 Oct  8 20:19 app.py
-rw----- 1 21 Oct  8 20:19 run.sh
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Nothing to undo.
```

```
# After undo: did anything come back?
$ ls -go Projects
total 8
-rw----- 1 12 Oct  8 20:19 app.py
-rw----- 1 21 Oct  8 20:19 run.sh
```

WITH THE ALGORITHM default configuration

```
# Initial state: which files are there?
$ ls -go Projects
total 8
-rw-r--r-- 1 12 Oct  8 20:19 app.py
-rwxr-xr-x 1 21 Oct  8 20:19 run.sh

# Ask SynapseOS, algorithm ON: the AI writes the command, then it runs
$ synapse 'run this exact command: find Projects -type f -exec chmod 600
{} +'
model: qwen2.5-coder:3b  endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: find Projects -type f -exec chmod 600 {} +
      stats: 27 tokens in 5.22s
blocked: find Projects -type f -exec chmod 600 {} + needs confirmation –
change metadata of /tmp/synapse-demo/Documents/SynapseOS-Playground/Pro
jects/app.py
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
      stats: 14 tokens in 6.352s
The command `find Projects -type f -exec chmod 600 {} +` was executed su
ccessfully, and the exit code was 0.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls -go Projects
total 8
-rw----- 1 12 Oct  8 20:19 app.py
-rw----- 1 21 Oct  8 20:19 run.sh
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: find Projects -type f -exec chmod 600 {} +
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:20:02+08:00)
restore permissions: /tmp/synapse-demo/Documents/SynapseOS-Playground/
Projects/app.py
restore permissions: /tmp/synapse-demo/Documents/SynapseOS-Playground/
Projects/run.sh
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls -go Projects
total 8
-rw-r--r-- 1 12 Oct  8 20:19 app.py
-rwxr-xr-x 1 21 Oct  8 20:19 run.sh
```

TC-05 Loop expansion

Pass

Request "run this exact command: for f in *draft.docx; do rm "\$f"; done" → model wrote `for f in *draft.docx; do rm "$f"; done` in both runs. Evidence: `ls *.docx` in the initial state, after execution, and after undo.

Observation Both runs removed exactly the two drafts and kept `thesis final.docx`. Only with the analysis did undo restore the drafts.

WITHOUT THE ALGORITHM SYNAPSE_ANALYSIS=off

```
# Initial state: which files are there?
$ ls *.docx
'resume draft.docx' 'thesis draft.docx' 'thesis final.docx'

# Ask SynapseOS, algorithm OFF: the AI writes the command, then it runs
$ SYNAPSE_ANALYSIS=off synapse 'run this exact command: for f in *draft.docx; do rm "$f"; done'
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: for f in *draft.docx; do rm "$f"; done
  stats: 29 tokens in 8.576s
blocked: for f in *draft.docx; do rm "$f"; done is irreversible – rm (rm
can permanently delete files with no built-in undo)
WARNING: This command decides what to delete while it runs, so I cannot
copy anything first. Undo will NOT be able to bring it back.
If you want the safety net, name the files or folders directly instead
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
  stats: 14 tokens in 10.505s
All draft.docx files have been deleted.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls *.docx
'thesis final.docx'
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: for f in *draft.docx; do rm "$f"; done
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:20:23+08:00)
note: 2 change(s) from this command could not be safely reconstructed
and are not part of this undo: [resume draft.docx thesis draft.docx]
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls *.docx
'thesis final.docx'
```

WITH THE ALGORITHM default configuration

```
# Initial state: which files are there?
$ ls *.docx
'resume draft.docx' 'thesis draft.docx' 'thesis final.docx'

# Ask SynapseOS, algorithm ON: the AI writes the command, then it runs
$ synapse 'run this exact command: for f in *draft.docx; do rm "$f"; done'
model: qwen2.5-coder:3b endpoint: http://localhost:11434
```

Each step may run for up to 2m0s before it's automatically stopped.

```
step 1: for f in *draft.docx; do rm "$f"; done
  stats: 29 tokens in 5.967s
blocked: for f in *draft.docx; do rm "$f"; done is irreversible – rm (rm
can permanently delete files with no built-in undo)
run it anyway? [y/N] y
exit code: 0
```

```
step 2: DONE
  stats: 14 tokens in 7.214s
All draft.docx files have been deleted.
Task complete in 1 step(s).
```

```
# After the command: what changed?
$ ls *.docx
'thesis final.docx'
```

```
# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: for f in *draft.docx; do rm "$f"; done
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T20:20:48+08:00)
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/r
esume draft.docx
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/t
hesis draft.docx
apply this undo? [y/N] y
Undo complete.
```

```
# After undo: did anything come back?
$ ls *.docx
'resume draft.docx' 'thesis draft.docx' 'thesis final.docx'
```

TC-06 Inherently irrecoverable operation

Pass

Request "run this exact command: shred -u "secret diary.txt"" → model wrote `shred -u "secret diary.txt"` in both runs. Evidence: `cat "secret diary.txt"` in the initial state, after execution, and after undo.

Observation Both runs required confirmation and neither could restore the file. `shred` destroys content by design, and the analysis does not claim otherwise.

WITHOUT THE ALGORITHM `SYNAPSE_ANALYSIS=off`

```
# Initial state: which files are there?
$ cat "secret diary.txt"
Dear diary,
Today I tested my own thesis.

# Ask SynapseOS, algorithm OFF: the AI writes the command, then it runs
$ SYNAPSE_ANALYSIS=off synapse 'run this exact command: shred -u "secret diary.txt"'
model: qwen2.5-coder:3b   endpoint: http://localhost:11434

Each step may run for up to 2m0s before it's automatically stopped.

step 1: shred -u "secret diary.txt"
  stats: 22 tokens in 5.566s
blocked: shred -u "secret diary.txt" is irreversible – shred (shred is designed to make file contents unrecoverable)
run it anyway? [y/N] y
exit code: 0

step 2: DONE
  stats: 14 tokens in 7.178s
The command shred -u "secret diary.txt" was run, and it successfully deleted the file "secret diary.txt".
Task complete in 1 step(s).

# After the command: what changed?
$ cat "secret diary.txt"
cat: 'secret diary.txt': No such file or directory

# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: shred -u "secret diary.txt"
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08 T20:21:03+08:00)
note: 1 change(s) from this command could not be safely reconstructed and are not part of this undo: [secret diary.txt]
apply this undo? [y/N] y
Undo complete.

# After undo: did anything come back?
$ cat "secret diary.txt"
cat: 'secret diary.txt': No such file or directory
```

WITH THE ALGORITHM default configuration

```
# Initial state: which files are there?
$ cat "secret diary.txt"
Dear diary,
Today I tested my own thesis.

# Ask SynapseOS, algorithm ON: the AI writes the command, then it runs
$ synapse 'run this exact command: shred -u "secret diary.txt"'
model: qwen2.5-coder:3b   endpoint: http://localhost:11434

Each step may run for up to 2m0s before it's automatically stopped.

step 1: shred -u "secret diary.txt"
  stats: 22 tokens in 4.387s
blocked: shred -u "secret diary.txt" is irreversible – shred (shred is designed to make file contents unrecoverable)
WARNING: This command decides what to delete while it runs, so I cannot copy anything first. Undo will NOT be able to bring it back.
If you want the safety net, name the files or folders directly instead.
run it anyway? [y/N] y
exit code: 0

step 2: DONE
  stats: 14 tokens in 5.467s
The command shred -u "secret diary.txt" was executed successfully, and the file "secret diary.txt" has been shredded and deleted.
Task complete in 1 step(s).

# After the command: what changed?
$ cat "secret diary.txt"
cat: 'secret diary.txt': No such file or directory

# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: shred -u "secret diary.txt"
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08 T20:21:22+08:00)
note: 1 change(s) from this command could not be safely reconstructed and are not part of this undo: [secret diary.txt]
apply this undo? [y/N] y
Undo complete.

# After undo: did anything come back?
$ cat "secret diary.txt"
cat: 'secret diary.txt': No such file or directory
```

TC-07 Explicitly named target (control case)

Pass

Request "delete the file named installer.zip" → model wrote `rm installer.zip` in both runs. Evidence: `ls *.zip` in the initial state, after execution, and after undo.

Observation Equivalent behaviour: both runs confirmed, backed up the named file, and restored it. The analysis's advantage is confined to commands whose targets are indirect.

WITHOUT THE ALGORITHM SYNAPSE_ANALYSIS=off

```
# Initial state: which files are there?
$ ls *.zip
installer.zip  photos-backup.zip

# Ask SynapseOS, algorithm OFF: the AI writes the command, then it runs
$ SYNAPSE_ANALYSIS=off synapse "delete the file named installer.zip"
model: qwen2.5-coder:3b  endpoint: http://localhost:11434

Each step may run for up to 2m0s before it's automatically stopped.

step 1: rm installer.zip
stats: 19 tokens in 4.036s
blocked: rm installer.zip is irreversible – rm (rm can permanently delete files with no built-in undo)
run it anyway? [y/N] y
exit code: 0

step 2: DONE
stats: 14 tokens in 5.34s
installer.zip has been deleted.
Task complete in 1 step(s).

# After the command: what changed?
$ ls *.zip
photos-backup.zip

# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: rm installer.zip
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08 T20:22:27+08:00)
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/installer.zip
apply this undo? [y/N] y
Undo complete.

# After undo: did anything come back?
$ ls *.zip
installer.zip  photos-backup.zip
```

WITH THE ALGORITHM default configuration

```
# Initial state: which files are there?
$ ls *.zip
installer.zip  photos-backup.zip

# Ask SynapseOS, algorithm ON: the AI writes the command, then it runs
$ synapse "delete the file named installer.zip"
model: qwen2.5-coder:3b  endpoint: http://localhost:11434

Each step may run for up to 2m0s before it's automatically stopped.

step 1: rm installer.zip
stats: 19 tokens in 3.895s
blocked: rm installer.zip is irreversible – rm (rm can permanently delete files with no built-in undo)
run it anyway? [y/N] y
exit code: 0

step 2: DONE
stats: 14 tokens in 5.109s
installer.zip has been deleted.
Task complete in 1 step(s).

# After the command: what changed?
$ ls *.zip
photos-backup.zip

# Undo: ask SynapseOS to reverse the last change
$ synapse undo
Undoing: rm installer.zip
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08 T20:22:41+08:00)
restore from trash: /tmp/synapse-demo/Documents/SynapseOS-Playground/installer.zip
apply this undo? [y/N] y
Undo complete.

# After undo: did anything come back?
$ ls *.zip
installer.zip  photos-backup.zip
```

TC-08 Composition ablation (analysis only)

Pass

Command `find . -name "* copy.jpg" -exec rm {} \;`, analysed with composition disabled

Expected With the same rule table but without composition, the analysis can no longer identify the targets.

```
# Ask the algorithm alone, with composition switched off (nothing runs)
$ $E -nocompose 'find . -name "* copy.jpg" -exec rm {} \;'
pattern list (today): asks first, with no backup (rm (rm can permanently delete files with no built
-in undo))
command:    find . -name "* copy.jpg" -exec rm {} \;
composition: disabled (table alone, ablation)
class:      unrecoverable
issue      opaque: composition disabled (ablation): find not resolved through
```

Observation. The command is classified unrecoverable with no targets identified, so the capture that restored the files in TC-02 cannot be planned. This mirrors Table 3.5: disabling composition reduces recovery coverage on the held-out corpus from 89.5% to 35.4%. Composition cannot be switched off in the deployed system, so this case examines the analysis alone.

PART B — EVERYDAY PHRASING

TC-09 repeats TC-02 with an everyday request, “delete the duplicate photos, the ones with copy in the name”. Its transcripts are long because the model makes several attempts, so each configuration is shown on its own page.

```
$ ls | grep copy
beach copy.jpg
sunset copy.jpg

$ SYNAPSE_ANALYSIS=off $S "delete the duplicate photos, the ones with co
py in the name"
model: qwen2.5-coder:3b   endpoint: http://localhost:11434

Each step may run for up to 2m0s before it's automatically stopped.

step 1: find . -type f -name '*copy*' -exec rm -f {} \
stats: 35 tokens in 58.74s
blocked: find . -type f -name '*copy*' -exec rm -f {} \ is irreversible
- rm (rm can permanently delete files with no built-in undo)
WARNING: This command decides what to delete while it runs, so I cannot
copy anything first. Undo will NOT be able to bring it back.
If you want the safety net, name the files or folders directly instead
.
run it anyway? [y/N] y
find: missing argument to `-exec'
exit code: 1

step 2: find . -type f -name '*copy*' -exec rm -f {} + 2>/dev/null
stats: 36 tokens in 15.339s
blocked: find . -type f -name '*copy*' -exec rm -f {} + 2>/dev/null is i
rreversible - rm (rm can permanently delete files with no built-in undo)
run it anyway? [y/N] y
exit code: 0

step 3: find . -type f -name '*copy*' -exec sh -c 'if [ $(grep -c
stats: 37 tokens in 16.397s
bash: -c: line 1: unexpected EOF while looking for matching `''
exit code: 2

This command already failed here, so running it again will not help: fin
d . -type f -name '*copy*' -exec sh -c 'if [ $(grep -c

$ ls | grep copy
(no copies left)

$ $S undo
Undoing: find . -type f -name '*copy*' -exec rm -f {} + 2>/dev/null
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08
T19:51:26+08:00)
restore content: /dev/null
apply this undo? [y/N] n
Cancelled.

$ ls | grep copy
(no copies left)
```

Observation. The copies were deleted at step 2. No warning that undo could not help was issued for that command, and the undo preview proposed only a spurious restoration of `/dev/null` (Issue 43). It was declined, and the copies were not recovered.

```

$ ls | grep copy
beach copy.jpg
sunset copy.jpg

$ $S "delete the duplicate photos, the ones with copy in the name"
model: qwen2.5-coder:3b  endpoint: http://localhost:11434

Each step may run for up to 2m0s before it's automatically stopped.

step 1: find . -type f -name '*copy*' -exec rm -f {} \
  stats: 35 tokens in 24.478s
blocked: find . -type f -name '*copy*' -exec rm -f {} \ is irreversible
- rm (rm can permanently delete files with no built-in undo)
  WARNING: This command decides what to delete while it runs, so I cannot
  copy anything first. Undo will NOT be able to bring it back.
  If you want the safety net, name the files or folders directly instead
.
run it anyway? [y/N] y
find: missing argument to `-exec'
exit code: 1

step 2: find . -type f -name '*copy*' -exec rm -f {} + 2>/dev/null
  stats: 36 tokens in 16.884s
blocked: find . -type f -name '*copy*' -exec rm -f {} + 2>/dev/null is irreversible
- rm (rm can permanently delete files with no built-in undo)
run it anyway? [y/N] y
exit code: 0

step 3: find . -type f -name '*copy*' -exec sh -c 'if [ $(grep -c
  stats: 37 tokens in 15.9s
blocked: find . -type f -name '*copy*' -exec sh -c 'if [ $(grep -c needs
confirmation - opaque: does not parse as shell: 1:43: reached EOF without
closing quote ``
  WARNING: This command decides what to delete while it runs, so I cannot
  copy anything first. Undo will NOT be able to bring it back.
  If you want the safety net, name the files or folders directly instead
.
run it anyway? [y/N] n
Cancelled.

$ ls | grep copy
(no copies left)

$ $S undo
Undoing: find . -type f -name '*copy*' -exec rm -f {} + 2>/dev/null
(ran in /tmp/synapse-demo/Documents/Synapse0S-Playground at 2026-10-08
T19:53:20+08:00)
  restore from trash: /tmp/synapse-demo/Documents/Synapse0S-Playground/beach copy.jpg
  restore from trash: /tmp/synapse-demo/Documents/Synapse0S-Playground/sunset copy.jpg
apply this undo? [y/N] y
Undo complete.

$ ls | grep copy
beach copy.jpg
sunset copy.jpg

$ cat "beach copy.jpg"
sample image data for beach copy.jpg

```

Observation. The same commands were generated. The copies were deleted at step 2; undo previewed their restoration from the trash and, once applied, both were present with their contents intact. A third, malformed command was flagged as unparseable and declined.

Action Invoking `synapse undo` again with no new action.

Expected Known limitation: `undo` targets the most recent journal entry, regardless of directory or session.

```
$ $S undo
Undoing: find . -type f -name '*copy*' -exec rm -f {} + 2>/dev/null
(ran in /tmp/synapse-demo/Documents/SynapseOS-Playground at 2026-10-08T19:11:46+08:00)
restore content: /dev/null
apply this undo? [y/N] n
Cancelled.
```

Observation. Undo proposed reverting an earlier entry and the proposal was declined. Correct use depends on the user reading the preview (Issue 42).

4. Findings

- 1. Without the analysis, data-destroying commands ran with no confirmation (TC-03, TC-04), consistent with the baseline's 41.2% silent-loss rate on the held-out corpus (§3.1(g)).
- 2. Where a command's targets are indirect (found at run time, or embedded in `find` or a loop), only the analysis made the deletion reversible: `undo` restored the files in TC-01, TC-02, TC-04, TC-05 and TC-09 with the analysis enabled, and in none of them with it disabled.
- 3. That capability is attributable to composition (TC-08; Table 3.5). Where targets are named directly, both configurations behave identically (TC-07).
- 4. Where effects cannot be determined or are irreversible by design, the analysis requires confirmation and does not claim recoverability (TC-03, TC-06).

5. Issues Identified

ISSUE	DESCRIPTION	SEVERITY
42	Undo is not scoped to the current context. It reverts the most recent journal entry, even from another directory or session; correct use depends on the user reading the preview.	Moderate; affects RQ3
43	The baseline path mishandles <code>2>/dev/null</code> . In the list-only configuration it is captured as an overwritten file and suppresses the warning that a deletion cannot be undone. It does not occur with the analysis enabled.	Low; baseline only

Both are recorded in `docs/open-problems.md` ; neither affects the corpus results.